SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1450 COURSE NAME: Compiler Design for Optimization

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

Total Marks:50

QUESTIONS:

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I N. Sonavarthi (192525083) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

N. Sonavarthi
Signature of the student:

Annexure A

ANALYTICAL PROBLEM SOLVING

1 Trace the input expression y = (p + q) * r - s / t through all phases of a compiler by identifying the tokens generated during lexical analysis, constructing the syntax tree during parsing, generating intermediate code, applying possible optimizations, and interpreting the relevance of the final target code produced.

2 Analyze the tokenization process of the expression float total = value--++ + ++count-- * 10.5; by identifying ambiguities in operator recognition, explaining how a LEX-based lexical analyzer processes such patterns, modifying the rules to resolve conflicts, and justifying the correctness of the updated lexical specification.

3 Determine the total number of tokens present in a given C program that includes nested loops, multiple variable declarations, arithmetic expressions, and inline comments, while carefully considering how the lexical analyzer ignores comments and whitespace. Analyze each line of the program to identify keywords, identifiers, operators, constants, and punctuation symbols as individual tokens, and compute the final token count based on proper lexical classification.

```
#include <stdio.h>

int main() {
    int i, j, sum = 0;    /* initialize variables */

    for(i = 0; i < 3; i++) {
        for(j = 0; j < 2; j++) {
            sum = sum + i * j;
        }
    }

    printf("%d\n", sum);
    return 0;
}
```

4 Analyze the performance of a lexical analyzer in a hypothetical programming language where the lexer processes up to 2,000,000 characters per second, with each token having an average length of 10 characters. Considering that the deterministic finite automaton (DFA) recognizes 25 different token types and performs an average of 2 state transitions per character, determine the total number of tokens generated, estimate the total number of DFA state transitions required, and interpret how efficiently the lexical analysis phase operates for the given source code size.

5 Analyze the working of a lexical analyzer for a simplified programming language that includes keywords (for, while, do, return, int, double), operators (+, -, *, /, =, ==, <=, >=), separators ((,), {, }, ;), identifiers (a sequence of letters followed by digits), integer literals, and floating-point literals. For the input string int var1 = 100; double var2 = 45.67; while (var1 <= var2) { var1 = var1 + 1; } return;, determine the total number of tokens generated by the lexical analyzer and provide a categorized list of tokens based on their types such as keywords, identifiers, operators, separators, and literals, ensuring proper classification and interpretation of each token.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Trace the expression through all phases of a compiler:

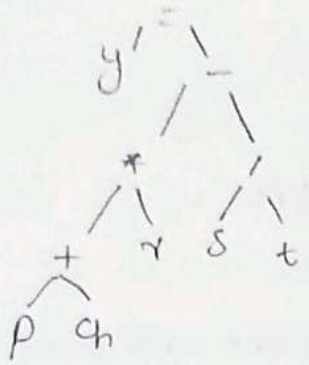
$y = (p + q) * r - s / t ;$

(a) Lexical Analysis (Tokens):

Lexeme	Token Type
y	Identifier
=	Assignment operator.
(	Left parenthesis
p	Identifier
+	Addition Operator
q	Identifier.
*	multiplication operator
r	Identifier
-	subtract operator
s	Identifier
/	division operator
t	Identifier
;	semicolon
)	right parenthesis.

Total tokens = 14

(b) Syntax Tree (parsing):



(c) Intermediate code (Three Address code).

$t_1 = p + q$

$t_2 = t_1 * r$

$t_3 = s / t$

$t_4 = t_2 - t_3$ (or) $r - s$

$y = t_4$.

(d) Optimization

No constant expression exist.

optimized Three Address code:

$t_1 = p + q$

$t_2 = t_1 * r$

$t_3 = s / t$

$y = t_2 - t_3$.

(c) Target code (Example)

(1)
```
LOAD R1, P
ADD R1, q
MUL R1, r
LOAD R2, s
DIV R2, t
SUB R1, R2
STORE y, R1
```

Interpretation :-

* Lexical analyzer generates tokens.
* parser builds syntax tree.
* Intermediate code is generated.
* optimization removes unnecessary instructions
* Target code executes on the processor.

(2) Tokenization of :-

float total = value--++ + ++count -- * 10.5;.

(a) Tokens

Lexeme	Token
float	keyword
total	identifier
=	Assignment
value	identifier
--	Decrement
++	increment
+	Addition
++	increment.
count	identifier
--	Decrement
*	multiplication.

10.8

;

floating constant

semicolon

(b) Ambiguity

The sequence

value++

could be interpreted as

value--

++

However

(value--)++

is invalid in C because the result of value-- is not an l-value.

Similarly;

++count--

means

(++count)--

(c) LEX Rules:-

"++" return INC;

"--" return DEC;

"==" return EQ;

"<=" return LE;

">=" return GE;

"=" return ASSIGN;

"+" return PLUS;

"-" return MINUS;

"*" return MUL;

"/" return DIV;

[0-9]+\.[0-9]+ return FLOAT;

[0-9]+ return INTEGER;

[a-zA-Z][a-zA-Z0-9]* return ID;

(2) Conflict Resolution:-

LEX always Choosed

* Longest matching token
* First rule when length are equal.

(3) Total tokens in the C program:-

```
#include <stdio.h>
int main(){
int i, j, sum=0;
for(i=0; i<3; i++){
for(j=0; j<2; j++){
sum = sum + i*j;
}
}
printf("%d\n", sum);
return 0;
}
```

Lexeme	Token
① # include	preprocessor Directive.
② <stdio.h>	Header file.

Token = 2
Running Total = 2

③ int	keyword
④ main	identifier
⑤ (	left parenthesis
⑥)	Right Parenthesis
⑦ {	Left Braces.
⑧ int	keyword.
⑨ i	identifier
⑩ ,	comma.
⑪ j	identifier.
⑫ ,	comma
⑬ sum	identifier.
⑭ =	Assignment Operator
⑮ 0	integer constant
;	semicolon.
for	keyword.
(	left Parenthesis
i	identifier.
=	Assignment
0	integer constant
;	semicolon.

i
<
3
;
i
++
)
(
for
(
j
=
0
;
j
<
2
;
j
++
)
{

identifier
Relational operator
integer constant
Semicolon
identifier.
increment operator
Right parenthesis
Left Brace.

keyword.
left parenthesis
identifier.
Assignment
integer constant.
semicolon.
identifier.
Relational operator.
integer constant
semicolon.
Identifier.
increment operator
Right Parenthesis
left Brace

sum

=

sum

+

i

*

j

;

}

}

printf

(

"%d/n"

)

sum

)

;

return

0

;

}

Identifier.

Assignment

Identifier.

Addition Operator.

Identified.

Multiplication Operator.

Identifier

Semicolon.

Right Brace

Right Brace

Identifier (Library function)

left Parenthis.

String Literal.

comma.

Identifier.

Right parenthesis.

Semicolon.

keyword.

Integere constant

Semicolon.

Right Brace.

Total Running = 65.

(4) (a) Performance of Lexical Analyzer.

Given

* Processing speed : 2,000,000 characters/sec
* Average token length = 10 characters.
* DFA transitions per character = 2.
* Token types = 25.

(a) Number of Tokens:

$$\frac{2000000}{10} \Rightarrow 200,000$$

Tokens generated = 200,000.

(b) DFA State Transitions:

Each character requires 2 transitions

2,000,000 × 2 = 4,000,000.

State transition = 4,000,000.

(c) Interpretation:

* character processed = 2,000,000
* Tokens recognized = 200,000
* DFA transitions = 4,000,000
* Token types supported = 25.

(5)

Tokens	Type
int	keyword
var1	Identifier
=	operator
100	integer literal
;	separator
double	keyword
var2	identifier
=	operator
45.67	floating literal
;	separator
while	keyword
(	separator
var1	identifier
<=	operator
var2	identifier
)	separator
{	separator
var1	Identifier
=	Identifier
var1	operator
+	identifier
1	operator
;	integer literal
}	separator
return	keyword

Total tokens = 26